

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 4.6 — Monitoring and Logging

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 4: Ensuring successful operation of a cloud solution (~20 % of the exam)**. You must know how to create Cloud Monitoring alerts, work with custom metrics, export logs, configure log routing, filter logs, use diagnostics, deploy the Ops Agent, deploy Managed Prometheus, and configure audit logs.

1. Cloud Monitoring Overview

What Is Cloud Monitoring?

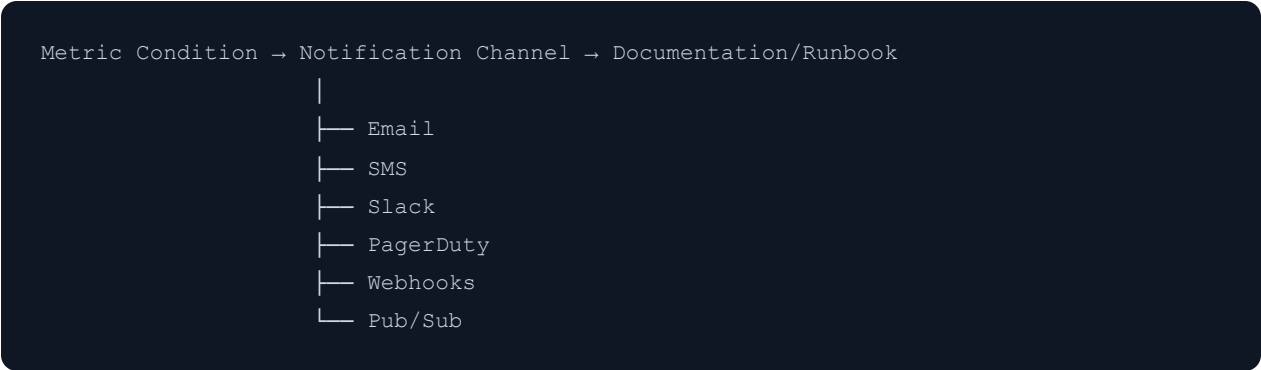
- Collects **metrics, events, and metadata** from Google Cloud, AWS, and on-premises
- Provides **dashboards, alerts, uptime checks**, and service-level monitoring
- Part of Google Cloud's Operations Suite (formerly Stackdriver)

Key Concepts

Concept	Description
Metric	A measured value over time (CPU utilization, request count, latency)
Time series	A sequence of metric data points
Resource	The entity being monitored (VM, GKE cluster, Cloud SQL instance)
Dashboard	Visual display of metrics and charts
Alert policy	Rules that trigger notifications based on metric conditions
Uptime check	Periodic checks that a service is available
SLO	Service Level Objective — target for a service's reliability

2. Creating Cloud Monitoring Alerts Based on Resource Metrics

Alert Policy Components



Creating Alerts via gcloud

```
# Create a notification channel (email)
gcloud alpha monitoring channels create \
  --display-name="Admin Email" \
  --type=email \
  --channel-labels=email_address=admin@example.com

# List notification channels
gcloud alpha monitoring channels list

# Create an alert policy (via JSON file)
gcloud alpha monitoring policies create \
  --policy-from-file=alert-policy.json
```

Alert Policy JSON Example

```
{
  "displayName": "High CPU Alert",
  "conditions": [
    {
      "displayName": "CPU utilization > 80%",
      "conditionThreshold": {
        "filter": "resource.type = \"gce_instance\" AND metric.type = \"compute.googleapis.com/cpu/usage\"",
        "comparison": "COMPARISON_GT",
        "thresholdValue": 0.8,
        "duration": "300s",
        "aggregations": [
          {
            "alignmentPeriod": "60s",
            "perSeriesAligner": "ALIGN_MEAN"
          }
        ]
      }
    }
  ],
  "notificationChannels": ["projects/PROJECT_ID/notificationChannels/CHANNEL_ID"],
  "documentation": {
    "content": "CPU is above 80%. Check for runaway processes or scale up.",
    "mimeType": "text/markdown"
  },
  "combiner": "OR",
  "enabled": true
}
```

Common Metrics for Alerts

Service	Metric	Description
Compute Engine	<code>compute.googleapis.com/instance/cpu/utilization</code>	CPU usage (0-1)
Compute Engine	<code>compute.googleapis.com/instance/disk/read_bytes_count</code>	Disk read bytes
Compute Engine	<code>compute.googleapis.com/instance/network/received_bytes_count</code>	Network bytes in
Cloud SQL	<code>cloudsql.googleapis.com/database/cpu/utilization</code>	Database CPU
Cloud SQL	<code>cloudsql.googleapis.com/database/memory/utilization</code>	Database memory
Cloud SQL	<code>cloudsql.googleapis.com/database/disk/utilization</code>	Disk usage
GKE	<code>kubernetes.io/container/cpu/request_utilization</code>	Container CPU vs request
GKE	<code>kubernetes.io/container/memory/used_bytes</code>	Container memory
Cloud Run	<code>run.googleapis.com/request_count</code>	Request count
Cloud Run	<code>run.googleapis.com/request_latencies</code>	Request latency
Load Balancer	<code>loadbalancing.googleapis.com/https/request_count</code>	LB request count
Load Balancer	<code>loadbalancing.googleapis.com/https/total_latencies</code>	LB latency

Uptime Checks

```
# Create an uptime check
gcloud monitoring uptime create my-uptime-check \
  --display-name="Website Check" \
  --resource-type=uptime-url \
  --hostname=www.example.com \
  --path=/health \
  --check-interval=60s \
  --timeout=10s
```

3. Custom Metrics

Creating Custom Metrics from Applications

Applications can write custom metrics to Cloud Monitoring:

Using the Monitoring API

```
from google.cloud import monitoring_v3
import time

client = monitoring_v3.MetricServiceClient()
project_name = f"projects/{project_id}"

# Create a time series data point
series = monitoring_v3.TimeSeries()
series.metric.type = "custom.googleapis.com/my_app/queue_depth"
series.resource.type = "global"

point = monitoring_v3.Point()
point.value.int64_value = 42
now = time.time()
point.interval.end_time.seconds = int(now)
series.points = [point]

# Write the data point
client.create_time_series(
    request={"name": project_name, "time_series": [series]}
)
```

Creating Custom Metrics from Logs (Log-Based Metrics)

Convert log entries into metrics:

```
# Create a counter metric (counts log entries matching a filter)
gcloud logging metrics create error-count \
  --description="Count of error log entries" \
  --log-filter='severity>=ERROR'

# Create a distribution metric (measures a value from log entries)
gcloud logging metrics create request-latency \
  --description="Request latency from access logs" \
  --log-filter='resource.type="gae_app"' \
  --bucket-bounds=0,100,200,500,1000,5000

# List log-based metrics
gcloud logging metrics list

# Describe a metric
gcloud logging metrics describe error-count

# Delete a metric
gcloud logging metrics delete error-count
```

Using Custom Metrics in Alerts

Custom metrics can be used in alert policies just like built-in metrics: - Filter: `metric.type = "custom.googleapis.com/my_app/queue_depth"` - Or for log-based: `metric.type = "logging.googleapis.com/user/error-count"`

4. Exporting Logs to External Systems

Log Export Destinations

Destination	Use Case
Cloud Storage	Long-term archival, compliance
BigQuery	SQL analysis, dashboards, reporting
Pub/Sub	Real-time processing, forward to on-premises/third-party
Cloud Logging bucket	Centralized log management, cross-project

Log Sinks

A **log sink** routes log entries matching a filter to a destination:

```
# Create a sink to Cloud Storage
gcloud logging sinks create my-storage-sink \
  storage.googleapis.com/my-log-bucket \
  --log-filter='severity>=WARNING'

# Create a sink to BigQuery
gcloud logging sinks create my-bq-sink \
  bigquery.googleapis.com/projects/PROJECT_ID/datasets/logs_dataset \
  --log-filter='resource.type="gce_instance"'

# Create a sink to Pub/Sub
gcloud logging sinks create my-pubsub-sink \
  pubsub.googleapis.com/projects/PROJECT_ID/topics/log-topic \
  --log-filter='severity>=ERROR'

# Create an aggregated sink (organization-level)
gcloud logging sinks create org-sink \
  storage.googleapis.com/org-logs-bucket \
  --organization=ORG_ID \
  --include-children \
  --log-filter='severity>=ERROR'

# List sinks
gcloud logging sinks list

# Describe a sink
gcloud logging sinks describe my-storage-sink

# Update a sink
gcloud logging sinks update my-storage-sink \
  --log-filter='severity>=ERROR'

# Delete a sink
gcloud logging sinks delete my-storage-sink
```

Sink Permissions

When you create a sink, Cloud Logging provides a **writer identity** (service account). You must grant this identity write access to the destination:

```
# Get the sink's writer identity
gcloud logging sinks describe my-bq-sink --format="get(writerIdentity)"

# Grant the writer identity access to the destination
# For BigQuery:
gcloud projects add-iam-policy-binding PROJECT_ID \
  --member="WRITER_IDENTITY" \
  --role="roles/bigquery.dataEditor"

# For Cloud Storage:
gcloud storage buckets add-iam-policy-binding gs://my-log-bucket \
  --member="WRITER_IDENTITY" \
  --role="roles/storage.objectCreator"
```

5. Configuring Log Buckets, Log Analytics, and Log Routers

Log Buckets

Cloud Logging stores logs in **log buckets**:

Bucket	Description	Retention
_Required	Admin activity, system event audit logs	400 days (not configurable)
_Default	All other logs	30 days (configurable)
Custom	User-created buckets	Configurable (1-3650 days)

```
# Create a custom log bucket
gcloud logging buckets create my-custom-bucket \
  --location=us-central1 \
  --retention-days=90 \
  --description="Custom log bucket for app logs"

# Update retention period
gcloud logging buckets update my-custom-bucket \
  --location=us-central1 \
  --retention-days=180

# List log buckets
gcloud logging buckets list

# Lock a bucket (prevent deletion and retention changes)
gcloud logging buckets update my-custom-bucket \
  --location=us-central1 \
  --locked
```

Log Analytics

Log Analytics lets you run SQL queries on logs stored in log buckets:

```
# Enable Log Analytics on a bucket
gcloud logging buckets update my-custom-bucket \
  --location=us-central1 \
  --enable-analytics

# Query logs using BigQuery SQL syntax in the Console
# Navigate to: Logging → Log Analytics
```

Log Router

The **Log Router** processes every log entry and routes it based on **inclusion and exclusion filters**:

```
Log Entry → Log Router
|
├─ _Required bucket (always)
├─ _Default bucket (unless excluded)
├─ Custom sinks (matching filters)
└─ Excluded (dropped, not stored)
```

Exclusion Filters

```
# Create an exclusion filter (stop specific logs from being stored)
gcloud logging sinks update _Default \
  --add-exclusion=name=exclude-debug,filter='severity=DEBUG'

# Exclude logs from a specific resource
gcloud logging sinks update _Default \
  --add-exclusion=name=exclude-lb-logs,filter='resource.type="http_load_balancer"'

# Remove an exclusion
gcloud logging sinks update _Default \
  --remove-exclusions=exclude-debug
```

6. Viewing and Filtering Logs in Cloud Logging

Reading Logs via gcloud

```
# Read recent logs
gcloud logging read --limit=50

# Read logs with a filter
gcloud logging read 'severity>=ERROR' --limit=20

# Read logs for a specific resource
gcloud logging read 'resource.type="gce_instance" AND resource.labels.instance_id="1234567" --limit=20

# Read logs for a time range
gcloud logging read 'timestamp>="2024-06-01T00:00:00Z" AND timestamp<="2024-06-02T00:00:00Z" --limit=100

# Read logs and format as JSON
gcloud logging read 'severity>=WARNING' --format=json --limit=10

# Read specific log name
gcloud logging read 'logName="projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2Factivities%2Fcompute.googleapis.com%2Fsystem%2Faudit"' --limit=10

# Combine filters
gcloud logging read '
  resource.type="gce_instance" AND
  severity>=ERROR AND
  textPayload:"connection refused"
' --limit=20
```

Common Log Filters

Filter	Description
<code>severity>=ERROR</code>	Error and Critical logs
<code>resource.type="gce_instance"</code>	Compute Engine VM logs
<code>resource.type="gke_cluster"</code>	GKE cluster logs
<code>resource.type="cloud_run_revision"</code>	Cloud Run logs
<code>resource.type="cloud_function"</code>	Cloud Functions logs
<code>resource.type="cloudsql_database"</code>	Cloud SQL logs
<code>logName="...cloudaudit.googleapis.com%2Factivity"</code>	Admin activity audit logs
<code>logName="...cloudaudit.googleapis.com%2Fdata_access"</code>	Data access audit logs
<code>protoPayload.methodName="v1.compute.instances.delete"</code>	Specific API method
<code>textPayload:"error message"</code>	Search in log text
<code>jsonPayload.status>=500</code>	JSON payload field

Log Entry Structure

```
{
  "logName": "projects/my-project/logs/compute.googleapis.com%2Factivity_log",
  "resource": {
    "type": "gce_instance",
    "labels": {
      "instance_id": "1234567890",
      "zone": "us-central1-a"
    }
  },
  "severity": "ERROR",
  "timestamp": "2024-06-15T10:30:00.000Z",
  "textPayload": "Connection refused on port 8080",
  "insertId": "abc123",
  "labels": {
    "compute.googleapis.com/resource_name": "my-vm"
  }
}
```

7. Using Cloud Diagnostics

Error Reporting

- Aggregates and displays errors from cloud services
- Groups identical errors together
- Shows error trends, first/last occurrence, affected users

```
# View error events
gcloud beta error-reporting events list

# Delete error events
gcloud beta error-reporting events delete
```

Cloud Trace

- Distributed tracing for latency analysis
- Shows how long each part of a request takes
- Automatically instruments App Engine, Cloud Run, Cloud Functions

Cloud Profiler

- Continuous CPU and memory profiling
- Low overhead (~0.5% CPU)
- Shows which functions consume the most resources

Viewing Google Cloud Status

- **Google Cloud Status Dashboard:** status.cloud.google.com
- Shows current status of all Google Cloud services
- Historical incidents and post-mortems
- Subscribe to RSS/Atom feeds for updates

8. Configuring and Deploying the Ops Agent

What Is the Ops Agent?

- **Unified agent** that collects both logs and metrics from VMs
- Replaces the legacy Monitoring and Logging agents
- Based on **Fluent Bit** (logs) and **OpenTelemetry Collector** (metrics)

- Supports custom log parsing and third-party application metrics

Installing the Ops Agent

```
# Install on a single VM via SSH
curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh
sudo bash add-google-cloud-ops-agent-repo.sh --also-install

# Verify the agent is running
sudo systemctl status google-cloud-ops-agent

# Install on multiple VMs using OS policies (VM Manager)
gcloud compute os-config os-policy-assignments create install-ops-agent \
  --location=us-central1-a \
  --file=ops-agent-policy.yaml
```

Configuring the Ops Agent

Configuration file: `/etc/google-cloud-ops-agent/config.yaml`

```
# Example: Collect nginx access logs and metrics
logging:
  receivers:
    nginx_access:
      type: nginx_access
    nginx_error:
      type: nginx_error
  service:
    pipelines:
      nginx:
        receivers:
          - nginx_access
          - nginx_error

metrics:
  receivers:
    nginx:
      type: nginx
      stub_status_url: http://localhost:8080/status
  service:
    pipelines:
      nginx:
        receivers:
          - nginx
```

```
# Restart the agent after configuration changes
sudo systemctl restart google-cloud-ops-agent
```

Supported Third-Party Applications

The Ops Agent has built-in support for: Apache, Nginx, MySQL, PostgreSQL, Redis, MongoDB, Elasticsearch, Cassandra, Kafka, RabbitMQ, and many more.

9. Deploying Managed Service for Prometheus

What Is Managed Prometheus?

- **Google-managed Prometheus** compatible monitoring
- Stores Prometheus metrics in Google Cloud (Monarch backend)
- Query with PromQL or Cloud Monitoring
- Drop-in replacement for self-managed Prometheus

Deployment Options

Option 1: Managed Collection (Recommended for GKE)

```
# Enable managed collection on a GKE cluster
gcloud container clusters update my-cluster \
  --zone=us-central1-a \
  --enable-managed-prometheus

# This deploys collection components automatically
```

```
# PodMonitoring resource to scrape your application
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata:
  name: my-app-monitoring
  namespace: default
spec:
  selector:
    matchLabels:
      app: my-app
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
```

Option 2: Self-Deployed Collection

```
# Deploy Prometheus with remote_write to Google Cloud
# In prometheus.yml
remote_write:
  - url: "https://monitoring.googleapis.com/v1/projects/PROJECT_ID/location/global/prometh
    # Authentication handled via Workload Identity
```

Querying with PromQL

- Available in Cloud Monitoring (Console → Monitoring → PromQL)
 - Use standard PromQL queries
 - Data is stored long-term in Google's Monarch backend
-

10. Configuring Audit Logs

Audit Log Types

Type	What It Logs	Enabled By Default	Charge
Admin Activity	API calls that modify resources (create, delete, update)	Yes (always on, cannot disable)	Free
Data Access	API calls that read data or metadata	No (must enable, except BigQuery)	Charged
System Event	Google-initiated actions (live migration, auto-scaling)	Yes (always on)	Free
Policy Denied	Security policy violations	Yes	Free

Enabling Data Access Audit Logs

```
# Get current audit config
gcloud projects get-iam-policy PROJECT_ID \
  --format=json > policy.json

# Edit policy.json to add audit configs:
```

```
{
  "auditConfigs": [
    {
      "service": "allServices",
      "auditLogConfigs": [
        {"logType": "ADMIN_READ"},
        {"logType": "DATA_READ"},
        {"logType": "DATA_WRITE"}
      ]
    },
    {
      "service": "storage.googleapis.com",
      "auditLogConfigs": [
        {"logType": "DATA_READ"},
        {"logType": "DATA_WRITE"}
      ]
    }
  ]
}
```

```
# Apply the updated policy
gcloud projects set-iam-policy PROJECT_ID policy.json
```

Viewing Audit Logs

```
# View admin activity logs
gcloud logging read 'logName="projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2Factivity" --limit=20

# View data access logs
gcloud logging read 'logName="projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2Fdata_access" --limit=20

# View who deleted a VM
gcloud logging read '
  logName="projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2Factivity" AND
  protoPayload.methodName="v1.compute.instances.delete"
' --limit=10

# View who changed IAM policies
gcloud logging read '
  logName="projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2Factivity" AND
  protoPayload.methodName="SetIamPolicy"
' --limit=10
```

Key Audit Log Fields

Field	Description
<code>protoPayload.authenticationInfo.principalEmail</code>	Who made the call
<code>protoPayload.methodName</code>	Which API method was called
<code>protoPayload.resourceName</code>	Which resource was affected
<code>protoPayload.status</code>	Result (success or error)
<code>timestamp</code>	When the call was made

8. SLOs and SLIs in Cloud Monitoring

- **SLI (Service Level Indicator)**: a quantitative measure of service behavior (e.g., request success rate, latency)
- **SLO (Service Level Objective)**: a target value for an SLI (e.g., 99.9% of requests succeed within 200ms)
- **Error budget**: $100\% - \text{SLO} = \text{allowable unreliability}$ (e.g., $0.1\% = \sim 43 \text{ min/month}$)

Creating a request-based SLO

```
# Requires a service to be created first (usually auto-discovered for GKE, Cloud Run, App
gcloud alpha monitoring services list
gcloud alpha monitoring slos create \
  --service=MY_SERVICE \
  --display-name="99.9% success rate" \
  --request-based \
  --good-total-ratio-threshold=0.999 \
  --method=GET \
  --path=/api/
```

SLO via Console: Monitoring → Services → select service → Create SLO → choose SLI type (Availability, Latency, or Custom metric) → set compliance period and goal.

- **Burn rate alerts**: alert when error budget is being consumed too fast
 - Fast burn (1h window, 14x burn rate): immediate response needed
 - Slow burn (6h window, 6x burn rate): investigation needed

9. Dashboard Creation

```
# List dashboards
gcloud monitoring dashboards list

# Create from JSON file
gcloud monitoring dashboards create --config-from-file=dashboard.json

# Delete
gcloud monitoring dashboards delete DASHBOARD_ID
```

Minimal dashboard JSON structure:

```
{
  "displayName": "My Service Dashboard",
  "gridLayout": {
    "widgets": [
      {
        "title": "Request Count",
        "xyChart": {
          "dataSets": [{
            "timeSeriesQuery": {
              "timeSeriesFilter": {
                "filter": "metric.type=\"run.googleapis.com/request_count\" resource.type="
              }
            }
          }]
        }
      }
    ]
  }
}
```

- In Console: Monitoring → Dashboards → Create Dashboard → Add Widget (Chart, Alert, Log panel, etc.)
- Pin charts from Metrics Explorer to dashboards

10. Advanced Log Filtering

MQL-style filter syntax (used in Log Explorer):

```
# Filter by severity
severity>=WARNING

# Filter by resource type and label
resource.type="gce_instance" AND resource.labels.zone="us-central1-a"

# Filter by log name
logName="projects/MY_PROJECT/logs/cloudaudit.googleapis.com%2Factivity"

# Filter by JSON payload field
jsonPayload.message=~"error" AND jsonPayload.userId="123"

# Time range
timestamp>="2024-06-01T00:00:00Z" AND timestamp<"2024-06-02T00:00:00Z"

# NOT operator
NOT textPayload:"health check"

# Cloud Run specific
resource.type="cloud_run_revision" AND resource.labels.service_name="my-service"

# GKE pod logs
resource.type="k8s_container" AND resource.labels.namespace_name="production" AND resource
```

Exam tip: Use `_Default` log bucket for the last 30 days; create custom log buckets with longer retention for compliance. Log exclusions reduce cost — exclude high-volume low-value logs (e.g., health check hits) at the log router before they're stored.

Exam Practice Questions

1. You need to be alerted when CPU utilization on any VM exceeds 80% for more than 5 minutes. How do you set this up?
 - Answer: Create a **Cloud Monitoring alert policy** with the metric `compute.googleapis.com/instance/cpu/utilization`, threshold > 0.8, duration 300 seconds, and configure a notification channel.
2. You need to track the number of HTTP 500 errors in your application logs. What's the best approach?
 - Answer: Create a **log-based metric** with a filter like `severity>=ERROR AND httpRequest.status=500`. Use this metric in dashboards and alerts.
3. Your compliance team requires all logs to be stored for 2 years. How do you configure this?

- Answer: Create a **custom log bucket** with `--retention-days=730` and route relevant logs to it using a log sink. Alternatively, export to **Cloud Storage** with appropriate retention.
4. **You need to export error logs to your on-premises SIEM system in real-time. How?**
- Answer: Create a **log sink** to a **Pub/Sub topic** with `--log-filter='severity>=ERROR'`. Configure the SIEM to subscribe to the Pub/Sub topic.
5. **What's the difference between the Ops Agent and the legacy Monitoring/Logging agents?**
- Answer: The Ops Agent is a **unified agent** that replaces both legacy agents. It's based on Fluent Bit (logs) and OpenTelemetry (metrics), supports more third-party applications, and is the recommended agent going forward.
6. **You need to investigate who deleted a Cloud Storage bucket. Where do you look?**
- Answer: Check **Admin Activity audit logs**: `gcloud logging read 'protoPayload.methodName="storage.buckets.delete"'`. Admin activity logs are always enabled and free.
7. **You want to use PromQL queries to monitor GKE workloads. What should you deploy?**
- Answer: Enable **Managed Service for Prometheus** on the GKE cluster with `gcloud container clusters update --enable-managed-prometheus`. Create PodMonitoring resources to scrape metrics.
-

Glossary

Admin Activity audit log: A Cloud Audit Log type that records all API calls that create, modify, or delete resources; always enabled, cannot be disabled, and is free of charge.

Alert policy: A Cloud Monitoring configuration that evaluates a metric condition and triggers notifications through one or more notification channels when the condition is met.

Alignment period: The time interval over which metric data points are aggregated (e.g., averaged) before being evaluated by an alert condition or displayed on a chart.

App Engine: Google Cloud's fully managed platform-as-a-service (PaaS) for building and deploying web applications; automatically instrumented by Cloud Trace.

Atom feed: A web feed format (alongside RSS) used to subscribe to updates from the Google Cloud Status Dashboard for service outage notifications.

Audit log: A record of who did what, when, and on which resource within Google Cloud; categorized into Admin Activity, Data Access, System Event, and Policy Denied types.

Burn rate alert: A Cloud Monitoring SLO alert that triggers when the error budget is being consumed at a rate that, if sustained, would exhaust the budget before the compliance period ends.

Cloud Functions: Google Cloud's serverless, event-driven compute service; automatically instrumented by Cloud Trace and natively writes logs to Cloud Logging.

Cloud Logging: Google Cloud's fully managed log ingestion, storage, search, and export service (part of the Operations Suite, formerly Stackdriver Logging).

Cloud Monitoring: Google Cloud's fully managed metrics collection, alerting, uptime checking, and dashboard service (part of the Operations Suite, formerly Stackdriver Monitoring).

Cloud Profiler: A Google Cloud service that continuously profiles CPU and memory usage of production applications with low overhead (~0.5%), identifying the most resource-consuming functions.

Cloud Run: Google Cloud's fully managed container execution environment for stateless HTTP services; automatically instrumented by Cloud Trace and writes logs to Cloud Logging.

Cloud Trace: Google Cloud's distributed tracing service that records how long each component of a request takes, used for latency analysis and performance optimization.

Compliance period: The time window over which an SLO is evaluated (e.g., rolling 30 days); used to calculate error budget consumption.

Custom log bucket: A user-created Cloud Logging storage bucket with configurable retention (1–3650 days), used for compliance, long-term storage, or workload-specific log isolation.

Custom metric: A user-defined metric written to Cloud Monitoring via the API or client library, used to track application-specific measurements not covered by built-in GCP metrics.

Dashboard: A Cloud Monitoring visual display of one or more charts, alert summaries, and log panels, used to observe the health and performance of cloud resources at a glance.

Data Access audit log: A Cloud Audit Log type that records API calls that read resource data or metadata; disabled by default (except BigQuery), and billed as log ingestion charges when enabled.

****_Default bucket**:** The built-in Cloud Logging log bucket that receives all logs not matched by other sinks; default retention is 30 days, which is configurable.

Error budget: The maximum allowable downtime or failure rate derived from an SLO (100% – SLO target); expressed in time or request count over the compliance period.

Error Reporting: A Google Cloud service that automatically aggregates, deduplicates, and surfaces application errors from Cloud Logging, showing trends and first/last occurrence.

Exclusion filter: A Cloud Logging Log Router rule that prevents specific log entries from being stored in a log bucket or forwarded to a sink, used to reduce log volume and cost.

Fluent Bit: A lightweight, high-performance log processor and forwarder; the log collection component of the Ops Agent.

GKE (Google Kubernetes Engine): Google Cloud's managed Kubernetes service; its clusters can be monitored using Cloud Monitoring built-in metrics, the Ops Agent, and Managed Service for Prometheus.

Google Cloud Status Dashboard: A public web page at status.cloud.google.com that displays real-time and historical availability information for all Google Cloud services.

IAM (Identity and Access Management): Google Cloud's access control system; used to grant the Ops Agent's VM the necessary permissions and to control who can create sinks, view logs, and manage alert policies.

ID token: A short-lived credential (up to 1 hour) issued by Google that identifies a service account, used for authenticating to services such as Cloud Run that verify Google-signed identity tokens.

Inclusion filter: The positive filter expression on a Cloud Logging sink that selects which log entries are forwarded to the sink's destination.

insertId: A field in a Cloud Logging log entry that provides a unique identifier for deduplication purposes.

jsonPayload: A structured Cloud Logging log entry field that holds log content in JSON format, allowing field-level filtering in queries.

Log Analytics: A Cloud Logging feature that enables SQL-based (BigQuery-syntax) queries over logs stored in analytics-enabled log buckets, directly in the Cloud Console.

Log bucket: The storage unit within Cloud Logging where log entries are stored; includes the built-in `_Required` and `_Default` buckets and any user-created custom buckets.

Log Explorer: The Cloud Console interface for searching, filtering, and viewing log entries in real time using the Cloud Logging query language.

Log Router: The Cloud Logging component that processes every incoming log entry and routes it to the appropriate log buckets and sinks based on inclusion and exclusion filters.

Log sink: A Cloud Logging configuration that exports log entries matching a filter to a destination such as Cloud Storage, BigQuery, Pub/Sub, or another log bucket.

Log-based metric: A Cloud Monitoring metric automatically derived from log entries matching a filter; can be a counter (counting matching entries) or a distribution (measuring a numeric value extracted from entries).

logName: A field in a Cloud Logging log entry that identifies which log stream the entry belongs to (e.g., `projects/PROJECT_ID/logs/cloudaudit.googleapis.com%2Factivity`).

Managed Service for Prometheus: Google Cloud's managed backend for storing and querying Prometheus metrics using PromQL, backed by Google's Monarch time-series database.

Metric: A measured, time-stamped value collected from a GCP resource (e.g., CPU utilization, request count, memory usage) and stored in Cloud Monitoring as a time series.

Monarch: Google's internal, globally distributed time-series database that serves as the backend for Cloud Monitoring and Managed Service for Prometheus.

MQL (Monitoring Query Language): A structured query language for advanced metric queries and alert conditions in Cloud Monitoring; distinct from the log filtering syntax used in Cloud Logging.

Notification channel: A Cloud Monitoring destination that receives alert notifications, including email, SMS, Slack, PagerDuty, Pub/Sub, and webhooks.

OIDC (OpenID Connect): An identity layer built on top of OAuth 2.0; used by Managed Service for Prometheus and Workload Identity Federation for authentication.

OpenTelemetry Collector: An open-source, vendor-neutral agent for collecting, processing, and exporting telemetry data (metrics, traces, logs); the metrics collection component of the Ops Agent.

Ops Agent: Google Cloud's recommended unified agent for collecting both logs and metrics from Compute Engine VMs, replacing the legacy Monitoring agent and Logging agent; based on Fluent Bit and OpenTelemetry Collector.

OS policies (VM Manager): A GCP feature that allows defining and enforcing desired OS configuration states across fleets of VMs, used to install agents like the Ops Agent at scale.

PagerDuty: A third-party incident management platform that can be configured as a Cloud Monitoring notification channel to receive and route alerts.

perSeriesAligner: A Cloud Monitoring aggregation setting that defines how individual data points within an alignment period are combined (e.g., `ALIGN_MEAN` averages them).

Policy Denied audit log: A Cloud Audit Log type that records when a request is denied by a security policy such as VPC Service Controls; enabled by default and free.

PodMonitoring: A Kubernetes custom resource (CRD) used with Managed Service for Prometheus to configure scraping of Prometheus metrics from specific pods in a GKE cluster.

PromQL (Prometheus Query Language): The query language used to query Prometheus-format metrics; supported natively in Managed Service for Prometheus and Cloud Monitoring.

protoPayload: A structured Cloud Logging log entry field that holds audit log data in Protocol Buffer format; contains fields like `methodName`, `resourceName`, and `authenticationInfo`.

Pub/Sub: Google Cloud's fully managed messaging service; used as a log sink destination for real-time forwarding of log entries to external SIEM systems or on-premises tools.

****_Required bucket****: The built-in Cloud Logging log bucket that stores Admin Activity and System Event audit logs with a fixed 400-day retention that cannot be changed or shortened.

resource.type: A field in a Cloud Logging log entry and monitoring filter that identifies the type of GCP resource that generated the entry (e.g., `gce_instance` , `cloud_run_revision`).

RSS feed: A web feed format used to subscribe to updates from the Google Cloud Status Dashboard for service outage notifications.

runbook: Documentation attached to a Cloud Monitoring alert policy that provides instructions for responding to the alert condition.

severity: A Cloud Logging field that classifies the importance of a log entry (e.g., `DEBUG` , `INFO` , `WARNING` , `ERROR` , `CRITICAL`); used as the primary field for log filtering.

SIEM (Security Information and Event Management): An external security platform that aggregates and analyzes log data; Cloud Logging can export logs to a SIEM via a Pub/Sub sink.

SLI (Service Level Indicator): A quantitative metric that measures a specific aspect of service behavior, such as request success rate or latency at a given percentile.

SLO (Service Level Objective): A target value or range for an SLI over a defined compliance period, representing the reliability goal for a service.

Stackdriver: The former name for Google Cloud's operations suite of monitoring, logging, tracing, and diagnostics services, now referred to as the Google Cloud Operations Suite.

System Event audit log: A Cloud Audit Log type that records Google-initiated system actions such as live migration, auto-scaling, and maintenance events; always enabled and free.

textPayload: A Cloud Logging log entry field that holds unstructured log content as a plain text string; used for keyword searches in log filters.

time series: An ordered sequence of metric data points, each associated with a timestamp, resource label, and metric label, stored and queried in Cloud Monitoring.

Uptime check: A Cloud Monitoring feature that periodically sends HTTP/HTTPS or TCP requests to a URL or IP address from multiple global locations to verify that a service is reachable.

VM Manager: A Google Cloud service for managing operating system configuration, patching, and inventory on Compute Engine VMs; used to deploy the Ops Agent at scale via OS policies.

Webhook: An HTTP callback that Cloud Monitoring can send to an external URL when an alert fires, enabling integration with custom alerting and incident management systems.

writer identity: A Google-managed service account automatically created by Cloud Logging for each log sink; must be granted write access to the sink's destination (e.g., BigQuery dataset, Cloud Storage bucket).

